INSTITUTO FEDERAL DO PARANÁ — CAMPUS IVAIPORÃ

CURSO DE BACHARELADO EM SISTEMAS DE INFORMAÇÃO

ATIVIDADE CURRICULAR DE EXTENSÃO (ACE)

JEAN CARLOS DOS SANTOS BANDURSKI

ARTHUR SCHIMANSKI VIEIRA

GUILHERME CORDEIRO DE GODOY

LUAN TRAINOTTI SILVA

THALYTA DA SILVA RODRIGUES

ANA CAROLINA SANCHES

DOCUMENTO DE ESPECIFICAÇÃO DE REQUISITOS DE SOFTWARE

Projeto de Extensão: Ivaí Conecta — plataforma de talentos, vagas e serviços locais

IVAIPORÃ — PR

2026

1 IDENTIFICAÇÃO E ESCOPO

Campo	Descrição
Sistema	Ivaí Conecta — aplicação web SPA.
Instituição	Instituto Federal do Paraná (IFPR) — Campus Ivaiporã.
Curso	Bacharelado em Sistemas de Informação — Atividade Curricular de Extensão.
Equipe	Jean Carlos dos Santos Bandurski; Arthur Schimanski Vieira; Guilherme Cordeiro de Godoy; Luan Trainotti Silva; Thalyta da Silva Rodrigues; Ana Carolina Sanches.
Abrangência	Ivaiporã e municípios do Vale do Ivaí; acesso web regional.
Base de análise	Código-fonte e configurações presentes no repositório recebido em 3 nov. 2026.

Este documento especifica os requisitos funcionais e não funcionais do Ivaí Conecta com base na implementação encontrada no repositório. A coluna de situação distingue recursos identificados no código de itens que dependem de configuração externa ou validação adicional. A especificação não presume que toda funcionalidade esteja habilitada no ambiente publicado.

2 VISÃO GERAL E AMBIENTE TECNOLÓGICO

O sistema é uma SPA desenvolvida em React 19, Vite 8 e React Router DOM 7. Utiliza Tailwind CSS 4, Zustand 5 e Firebase JavaScript SDK 12, com Firebase Authentication, Cloud Firestore e Cloud Storage. O cliente também consome BrasilAPI para consulta de CNPJ e abre links externos do WhatsApp. Não foi identificada API própria em servidor, Cloud Functions, integração de IA generativa ou banco relacional no pacote principal.

Componente	Tecnologia/versão declarada	Uso
Linguagem	JavaScript ES Modules / JSX	Interface e regras de negócio.
Interface	React / React DOM ^19.2.8	Componentes e páginas.
Build e desenvolvimento	Vite ^8.2.0; @vitejs/plugin-react ^6.0.4	Servidor local e compilação.
Rotas	React Router DOM ^7.18.2	Navegação e proteção de rotas.
Estilos	Tailwind CSS ^4.3.3; @tailwindcss/forms	Estilização responsiva.
Persistência remota	Firebase ^12.17.1	Authentication, Firestore e Storage.
Estado	Zustand ^5.0.15	Estado global.
Ícones	Lucide React ^1.32.0	Ícones da interface.
Documentos	jsPDF ^4.2.1; html2canvas-pro ^2.5.0	Geração/captura de PDF.
Qualidade	Oxlint ^1.75.0	Análise estática.

3 ATORES E PERFIS

Ator	Responsabilidades e permissões esperadas
Visitante	Acessar páginas públicas, consultar vagas, serviços e perfis com visibilidade pública.
Aluno	Gerenciar perfil acadêmico/profissional, consultar vagas, candidatar-se, solicitar endosso e acompanhar candidaturas.
Candidato	Gerenciar perfil profissional, consultar vagas e candidatar-se.
Freelancer	Gerenciar perfil de prestador, publicar serviços, acompanhar pedidos e avaliações.
Empresa	Gerenciar perfil empresarial e vagas, consultar candidatos,

	movimentar etapas e agendar entrevistas.
Professor	Manter perfil docente e avaliar solicitações de endosso.
Administrador	Gerenciar usuários, vagas, serviços, moderação, indicadores e configurações administrativas.

4 REQUISITOS FUNCIONAIS

ID	Descrição do requisito funcional	Prioridade	Situação observada
RF01	O sistema deve permitir cadastro de conta com os papéis aluno, candidato, freelancer, empresa e professor.	Essencial	Implementado no fluxo de cadastro; regras específicas por perfil.
RF02	O sistema deve permitir autenticação por e-mail/senha e autenticação com Google quando o provedor estiver habilitado no Firebase.	Essencial	Implementado; Google depende de configuração externa.
RF03	O sistema deve permitir encerramento de sessão e observar alterações no estado de autenticação.	Essencial	Implementado.
RF04	O sistema deve permitir complementação e edição dos dados do perfil, incluindo dados pessoais, formação, experiências, projetos, competências e informações profissionais aplicáveis.	Essencial	Implementado por páginas e serviços de perfil.
RF05	O sistema deve permitir validação de e-mail institucional/estudantil e consulta do estado de verificação da conta.	Importante	Implementado; validação efetiva depende do Firebase Authentication e domínios aceitos.
RF06	O sistema deve permitir consulta e validação de CNPJ durante o cadastro/gestão empresarial.	Importante	Implementado com BrasilAPI e validação local.
RF07	O sistema deve permitir consulta e filtragem do banco de talentos e visualização de perfis individuais.	Essencial	Implementado.
RF08	O sistema deve permitir que estudantes solicitem endossos/recomendações a docentes e que docentes aprovem ou recusem solicitações.	Importante	Implementado por solicitações e serviços de endosso.
RF09	O sistema deve permitir a consulta, pesquisa e filtragem de vagas por critérios disponíveis, incluindo texto, localidade e características da oportunidade.	Essencial	Implementado.
RF10	O sistema deve permitir que empresas criem, editem, dupliquem, alterem o status e excluam vagas sob sua responsabilidade.	Essencial	Implementado nos serviços e painel empresarial.
RF11	O sistema deve permitir que candidatos consultem detalhes de vagas e enviem candidaturas, impedindo candidatura incompatível com as regras de elegibilidade.	Essencial	Implementado.
RF12	O sistema deve permitir que candidatos consultem suas candidaturas e desistam/cancelem quando permitido pelo estado do processo.	Essencial	Implementado conforme estados e regras.
RF13	O sistema deve permitir que empresas visualizem candidatos e gerenciem etapas do processo seletivo, incluindo movimentação de etapa, atualização de dados e agendamento de entrevista.	Essencial	Implementado no gerenciamento empresarial.
RF14	O sistema deve calcular e apresentar um indicador de	Importante	Implementado em matchService.js; algoritmo

	compatibilidade entre competências do perfil e requisitos da vaga.		determinístico, não IA generativa.
RF15	O sistema deve permitir consulta, busca e filtragem de serviços locais por informações disponíveis, como categoria e localidade.	Essencial	Implementado.
RF16	O sistema deve permitir que freelancers publiquem, editem e excluam anúncios de serviços e abram contato externo por WhatsApp.	Essencial	Implementado; contato usa link wa.me.
RF17	O sistema deve permitir que clientes registrem solicitações de serviços e que prestadores atualizem o estado dos pedidos.	Importante	Implementado por serviços de pedidos.
RF18	O sistema deve permitir o envio de avaliações de serviços e a atualização dos dados agregados de avaliação.	Importante	Implementado em servicesService.
RF19	O sistema deve permitir comunicação entre usuários por conversas e mensagens, com indicação de mensagens não lidas.	Importante	Implementado em chatService e ChatModal.
RF20	O sistema deve permitir favoritar e desfavoritar vagas e perfis de talentos.	Desejável	Implementado para vagas e talentos.
RF21	O sistema deve disponibilizar área administrativa restrita para consulta de indicadores, gestão de usuários, vagas e serviços e moderação.	Essencial	Implementado; requer papel admin e configuração segura.
RF22	O sistema deve permitir upload de imagem de perfil e documentos suportados para armazenamento remoto.	Importante	Implementado no Storage; regras e tipos/tamanhos devem ser verificados.
RF23	O sistema deve permitir geração/visualização de currículo em PDF quando a opção estiver disponível no perfil.	Desejável	Implementado no componente ResumePDFModal.
RF24	O sistema deve manter dados auxiliares em cache local para melhorar a experiência e suportar alguns fluxos de contingência.	Desejável	Parcial; cache local não equivale a operação offline integral.

5 REQUISITOS NÃO FUNCIONAIS

ID / categoria	Descrição	Situação	Evidência/observação
RNF01 — Desempenho	A interface deve carregar recursos de forma eficiente, utilizando divisão de código e carregamento sob demanda quando aplicável.	Parcial/validar	Há lazy loading de páginas; não há medição de tempo que comprove meta fixa de 2 segundos.
RNF02 — Responsividade	As telas devem adaptar-se a smartphones, tablets e desktops, mantendo acesso às funções essenciais.	Implementado no código; validar	Tailwind CSS e estilos responsivos presentes; requer teste em dispositivos reais.
RNF03 — Compatibilidade	A aplicação deve funcionar em versões atuais de Chrome, Edge, Firefox e Safari, além de navegadores móveis compatíveis com o Firebase SDK.	Dependente de validação	A compatibilidade depende das versões dos navegadores e dos recursos Firebase usados.
RNF04 — Segurança de autenticação	A aplicação deve utilizar Firebase Authentication e proteger rotas e operações conforme o papel e a identidade do usuário.	Parcial	Há rotas protegidas e regras Firestore; a segurança final depende também da revisão das regras e configuração remota.
RNF05 — Privacidade	Dados pessoais, documentos e conversas devem ser acessíveis apenas a usuários autorizados, aplicando o princípio do menor privilégio e observando a LGPD.	Pendente de revisão	Regras fornecidas incluem leitura pública em users e Storage; avaliar exposição e restringir acesso.
RNF06 — Integridade	Operações de escrita devem validar propriedade, estado e	Parcial	Existem validações em serviços e regras; testes de segurança

	consistência dos dados no cliente e, principalmente, nas regras de segurança remotas.		abrangentes não foram identificados.
RNF07 — Manutenibilidade	O código deve manter separação entre páginas, componentes, serviços, repositórios, modelos e estado compartilhado.	Implementado na estrutura	Diretórios e módulos correspondentes estão presentes.
RNF08 — Disponibilidade	A disponibilidade dos dados e da autenticação depende dos serviços Firebase e da hospedagem configurada.	Dependente de terceiros	Não há SLA próprio no repositório; disponibilidade não deve ser garantida pela aplicação.
RNF09 — Acessibilidade e usabilidade	Formulários, navegação, mensagens de erro e controles devem ser compreensíveis e utilizáveis por diferentes públicos.	Validar	Há componentes e feedbacks de interface; não foi localizada auditoria formal WCAG.
RNF10 — Testabilidade	O projeto deve disponibilizar verificações automatizadas para reduzir regressões em regras de negócio e interface.	Parcial	Há lint, check:mocks e test:cbo; não foi identificada suíte ampla de testes unitários/e2e.
RNF11 — Configuração e implantação	A aplicação deve permitir configuração por variáveis de ambiente e build reproduzível por npm.	Implementado com ressalvas	Há package-lock e variáveis VITE_FIREBASE_*; firebase.json não define Hosting.
RNF12 — Armazenamento de arquivos	Uploads devem respeitar autenticação, propriedade, tamanho e tipos permitidos, sem exposição indevida.	Parcial/validar	Upload de perfil/documento existe; regras Storage permitem leitura pública e exigem revisão.

6 REGRAS DE NEGÓCIO E RESTRIÇÕES

- Os papéis de usuário reconhecidos no modelo são aluno, candidato, freelancer, empresa, professor e admin. A atribuição de administrador não deve ser realizada pelo próprio usuário.
- Contas de estudante e docente passam por verificações específicas de e-mail institucional; os domínios aceitos são definidos no serviço de autenticação/verificação.
- O cadastro empresarial valida a estrutura do CNPJ e consulta dados externos quando o serviço está disponível.
- Uma candidatura deve estar associada a um candidato e a uma vaga; o recrutador administra as etapas e o candidato não pode atribuir a si próprio aprovação.
- Endossos são solicitados pelo estudante e avaliados pelo docente associado à solicitação; o estudante não pode aprovar o próprio pedido.
- Serviços publicados pertencem ao prestador responsável; avaliações e contadores não devem ser manipulados livremente pelo proprietário do anúncio.
- Conversas e mensagens devem respeitar a participação dos usuários envolvidos.
- Os recursos externos (Firebase, BrasilAPI, WhatsApp e Google Fonts) dependem de conectividade e disponibilidade dos respectivos provedores.

7 DADOS E INTEGRAÇÕES

Coleção / serviço	Finalidade identificada

users	Perfis, papéis, dados de conta e estado de verificação.
jobs	Vagas publicadas por empresas.
applications	Candidaturas e etapas do processo seletivo.
services	Anúncios de serviços, dados de avaliação e métricas.
endorsement_requests	Solicitações e pareceres docentes.
favorites	Vagas e talentos favoritos por usuário.
chats / messages	Conversas e mensagens entre participantes.
service_orders	Solicitações e acompanhamento de pedidos de serviço.
talents / teachers	Dados auxiliares de talentos e docentes usados pelos serviços.
Firebase Storage	Imagens de perfil e documentos enviados.
BrasilAPI / wa.me	Consulta de dados de CNPJ e abertura de contato externo, respectivamente.

8 TELAS E ROTAS IDENTIFICADAS

Rota	Tela / função
/	Dashboard / página inicial e destaques.
/login	Cadastro, autenticação e fluxos de acesso.
/completar-perfil	Complementação de perfil.
/vagas	Consulta de oportunidades.
/servicos	Catálogo de serviços locais.
/talentos	Banco de talentos.
/perfil e /perfil/:id	Perfil do estudante/candidato.
/empresa e /empresa/:id	Perfil empresarial.
/professor e /professor/:id	Perfil docente.
/freelancer e /freelancer/:id	Perfil do prestador.
/configuracoes	Configurações da conta.
/minhas-vagas	Candidaturas do usuário ou gestão empresarial conforme papel.
/gerenciamento-vagas e /painel-empresa	Painel de gestão de vagas.
/admin	Dashboard administrativo.
/admin/users, /admin/jobs, /admin/services	Gestão de usuários, vagas e serviços.
/admin/moderation, /admin/settings	Moderação e configurações administrativas.

9 REQUISITOS DE AMBIENTE

Para desenvolvimento, utilizar Node.js compatível com a versão do Vite declarada no package.json (Vite 8: Node.js 20.19+ ou 22.12+), npm e Git. A instalação deve ser realizada no diretório App com npm ci, seguida de npm run dev para desenvolvimento ou npm run build para compilação. O usuário final necessita de navegador atualizado, JavaScript habilitado e conexão HTTPS.

As variáveis VITE_FIREBASE_API_KEY, VITE_FIREBASE_AUTH_DOMAIN, VITE_FIREBASE_PROJECT_ID, VITE_FIREBASE_STORAGE_BUCKET, VITE_FIREBASE_MESSAGING_SENDER_ID, VITE_FIREBASE_APP_ID e VITE_FIREBASE_MEASUREMENT_ID são reconhecidas pelo inicializador Firebase. Valores VITE_* são enviados ao cliente e não devem conter segredos administrativos.

10 RISCOS E PENDÊNCIAS DE HOMOLOGAÇÃO

- Revisar regras Firestore e Storage: as regras fornecidas autorizam leitura pública de perfis de usuários e de objetos do Storage. Definir quais campos e arquivos podem ser públicos e restringir documentos privados.
- Revisar a regra de listagem da coleção chats, que permite listagem a usuários autenticados; assegurar que consultas e regras impeçam exposição de conversas de terceiros.
- Executar testes de autorização para cada papel, especialmente atualização de perfil, candidaturas, etapas seletivas, pedidos, avaliações, documentos e funções administrativas.
- Configurar e validar Firebase Authentication, Firestore, Storage, índices e domínio de hospedagem no ambiente de destino.
- Não tratar cache local como modo offline completo; validar comportamento com perda de conexão e sincronização posterior.
- Adicionar testes automatizados de unidade, integração e ponta a ponta para os fluxos críticos.
- Validar acessibilidade, responsividade, desempenho e comportamento em navegadores e dispositivos reais.
- Documentar política de privacidade, retenção, exclusão e atendimento aos direitos dos titulares de dados conforme LGPD.

11 REFERÊNCIAS

BANDURSKI, Jean Carlos dos Santos et al. Ivaí Conecta: plataforma web regional de talentos, oportunidades de trabalho e serviços locais. Repositório de software. GitHub, 2026. Disponível em: https://github.com/JeanBandurski-Dev/ivai-conecta. Acesso em: 3 nov. 2026.

GOOGLE. Firebase Documentation. Disponível em: https://firebase.google.com/docs. Acesso em: 3 nov. 2026.

REACT. React Documentation. Disponível em: https://react.dev. Acesso em: 3 nov. 2026.

VITE. Vite Documentation. Disponível em: https://vite.dev/guide/. Acesso em: 3 nov. 2026.